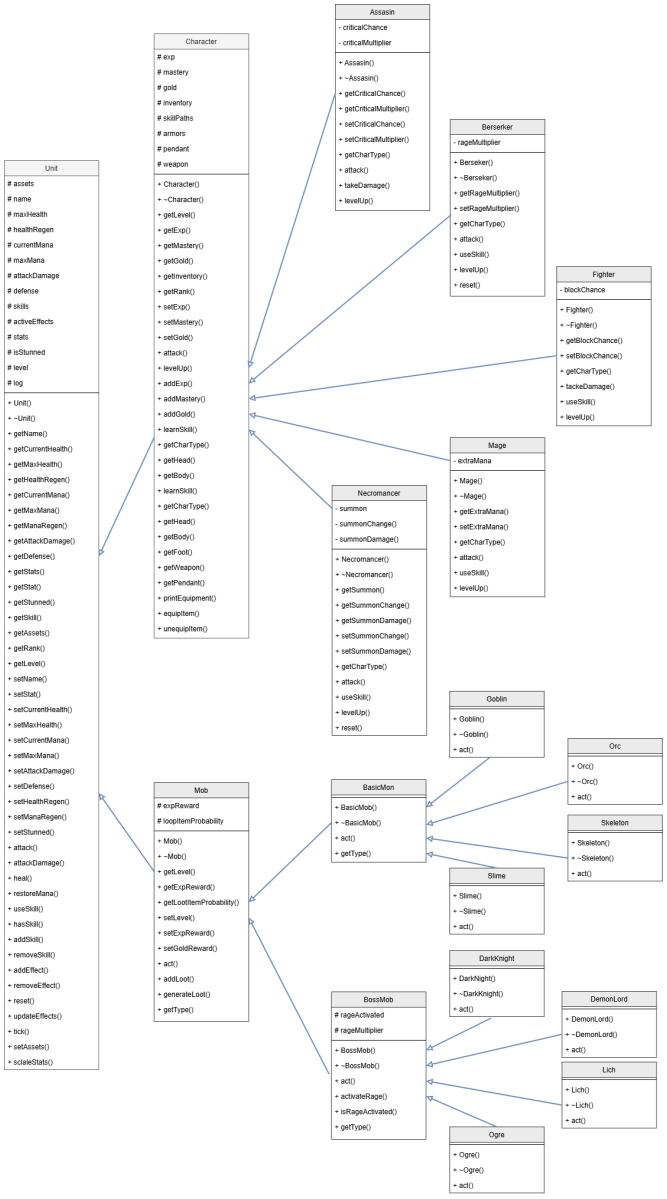
Kode Kelompok : RPL
Nama Kelompok : akuAdalahOOP
1. 13522051 / Kharris Khisunica
2. 13523073 / Alfian Hanif Fitria Yustanto
3. 13523091 / Carlo Angkisan
4. 13523107 / Heleni Gratia M Tampubolon
5. 13523121 / Ahmad Wicaksono
Asisten Pembimbing : 13523135 / Nicholas Liem

1. Diagram Kelas

Diagram Kelas	Alasan Pemilihan



Pada desain diagram kelas ini, dipilih pendekatan pewarisan (*inheritance*) dengan menjadikan Unit sebagai kelas induk (superclass) yang memiliki dua kelas turunan, yaitu Character dan Mob. Desain ini dipilih karena baik Character maupun Mob memiliki atribut dan perilaku dasar yang serupa seperti name, manaRegen, attackDamage dan metode seperti updateEffects() dan scaleStats().

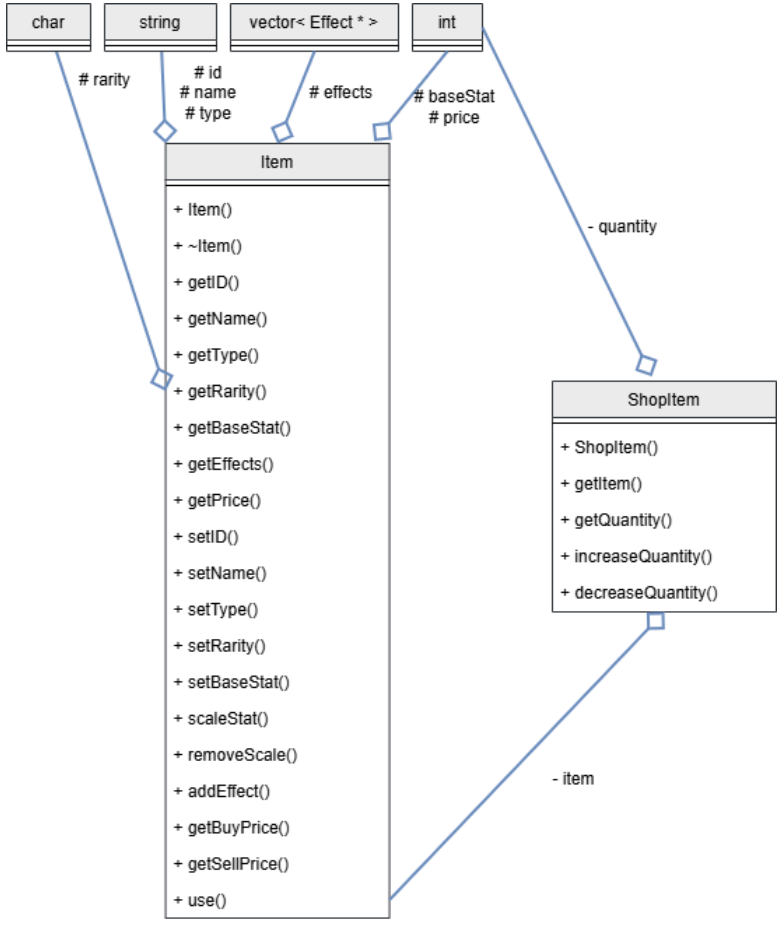
Untuk mendukung keragaman karakter dan musuh dalam game, hierarki diturunkan lebih lanjut. Character memiliki subclass spesifik yang merepresentasikan tipe-tipe karakter, yaitu Assassin, Berserker, Fighter, Mage, dan Necromancer. Untuk Mob diturunkan menjadi dua jenis, yaitu BasicMob dan BossMob. Kedua kelas ini juga memiliki turunan lebih lanjut untuk membedakan tipe musuh yang lebih spesifik, seperti Slime, Orc, dan Lich.

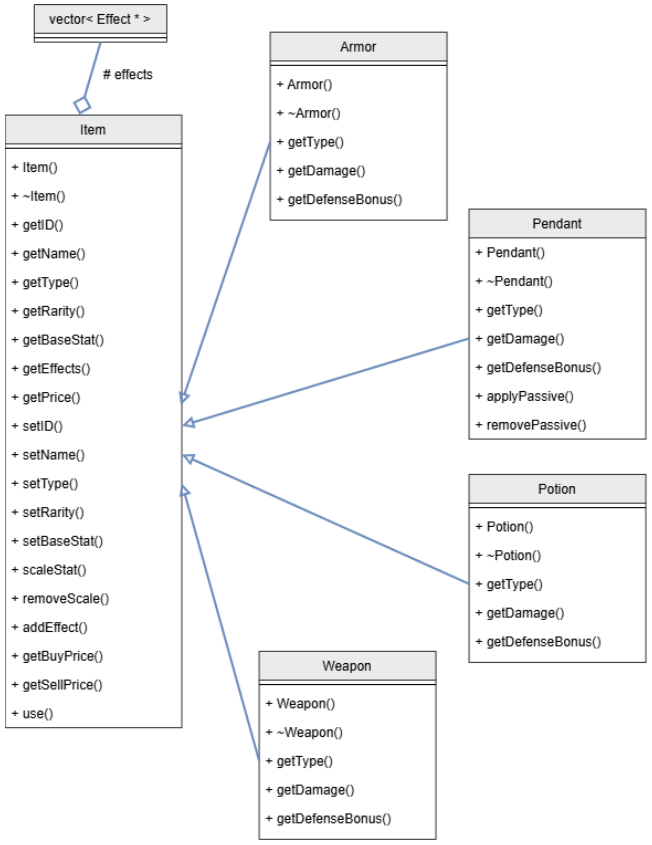
Setiap subclass dapat memiliki atribut dan perilaku tambahan yang lebih spesifik sesuai kebutuhan peran atau karakteristiknya. Desain ini mendukung prinsip DRY (Don’t Repeat Yourself) sehingga kode lebih mudah dikelola dan dikembangkan.
Kelebihan:

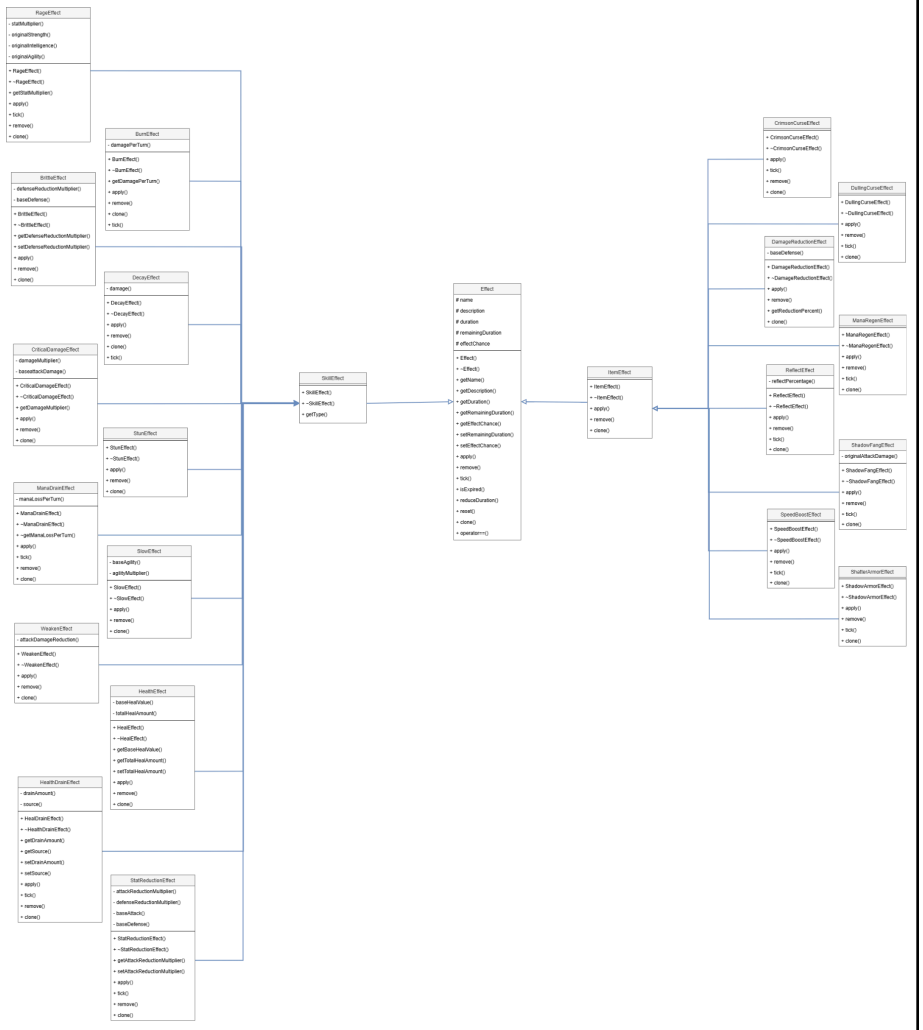
1. Reusability, fungsi-fungsi dasar seperti updateEffects() hanya perlu ditulis sekali di kelas Unit.
2. Fleksibilitas, memudahkan pengembangan fitur spesifik masing-masing subclass (Character bisa memiliki exp, mastery, gold dan Mob bisa memiliki expReward dan goldReward).
3. Struktur yang rapi, memisahkan tanggung jawab kelas secara jelas.

Kekurangan:

1. Keterikatan hierarkis, jika terlalu bergantung pada inheritance, bisa menyulitkan ketika ingin mengubah struktur atau perilaku umum di Unit.
2. Overhead abstraksi, jika unit terlalu beragam dan berbeda yang menyebabkan inheritance bisa menjadi tidak efisien dan malah membingungkan.

	Kendala yang Dihadapi: 1. Menentukan generalisasi yang tepat. Di awal sedikit sulit menentukan atribut dan metode mana yang sebaiknya dimiliki Unit dan mana yang hanya relevan untuk Character atau Mob.
	Desain ini menggunakan pendekatan agregasi untuk membangun relasi antara ShopItem dan Item, serta antara Item dengan tipe data dasar (string, char, int, dan vector<Effect*>). Kelas Item bertindak sebagai entitas utama yang merepresentasikan item dalam sistem,sementara ShopItem adalah representasi item yang menyertakan kuantitas. Desain ini dipilih karena memisahkan tanggung jawab antara data dan logika item (Item) dan konteks toko yang berisi berbagai item (ShopItem). Kelebihan: 1. Modularitas tinggi, hal ini disebabkan Item dan ShopItem dipisahkan secara jelas sehingga dapat digunakan di berbagai konteks (shop, inventory) tanpa modifikasi. 2. Reusability, item dapat digunakan secara mandiri dalam kombinasi dengan entitas lain (tidak bergantung pada ShopItem dan sebaliknya juga), seperti penggunaannya untuk inventory. Kekurangan: 1. Abstraksi bertingkat, banyaknya level abstraksi (proses menyembunyikan detail implementasi yang tidak perlu dan hanya menampilkan informasi yang penting/relevan) dari ShopItem → Item → Effect dapat membingungkan jika tidak terdokumentasi dengan baik. 2. Pengelolaan memori, disebabkan penggunaan pointer (Effect*) menimbulkan beban manajemen memori manual (heap allocation dan deallocation). Kendala yang Dihadapi: 1. Penentuan kepemilikan. Awalnya bisa membingungkan apakah ShopItem harus mewarisi Item, atau menyimpan

<table>
<tr>
<td></td>
<td>referensi ke Item. Pemilihan agregasi dilakukan agar desain lebih fleksibel.
2. Penggunaan efek. Menentukan bagaimana efek-efek ditambahkan dan dikaitkan ke item membutuhkan desain Effect yang fleksibel.
3. Harga dan Statistik. Keputusan untuk menaruh price di Item menandakan bahwa harga bukan dipengaruhi toko tetapi dimiliki secara independen oleh item.</td>
</tr>
<tr>
<td> 
</td>
<td>Pada desain diagram kelas ini, dipilih pendekatan pewarisan (inheritance) dengan menjadikan Item sebagai kelas induk (superclass) yang memiliki empat kelas turunan, yaitu Armor, Weapon, Potion, daan Pendant. Desain ini dipilih karena kelas turunan tersebut memiliki atribut dan perilaku dasar yang serupa seperti name, id, type, dan effect. Terdapat juga atribut vector<Effect*> yang merupakan agregasi dari Item yang menunjukkan bahwa sebuah item dapat memiliki banyak Effect.
Kelebihan:
1. Modularitas, bahwa semua jenis item tersusun rapi di bawah satu struktur Item, memudahkan pemeliharaan dan pengembangan.
2. Fleksibilitas, memudahkan pengembangan fitur spesifik pada jenis item masing-masing subclass.
3. Reusability, beberapa method cukup didefinisikan sekali di Item, lalu digunakan di subclass sesuai dengan kebutuhan.
Kekurangan:
1. Keterikatan hierarkis, jika terlalu bergantung pada inheritance, bisa menyulitkan ketika ingin mengubah struktur atau perilaku umum di Item.</td>
</tr>
</table>



Pada desain diagram kelas ini, digunakan pendekatan pewarisan (*inheritance*) dengan menjadikan Effect sebagai kelas induk (*superClass*) yang memiliki dua kelas turunan, yaitu SkillEffect dan ItemEffect. Pendekatan ini dipilih karena baik SkillEffect maupun ItemEffect memiliki atribut dan perilaku dasar yang serupa, seperti name, duration, effectChance, serta metode seperti getEffectChance() dan getDuration().

Untuk mendukung keragaman karakter dan musuh dalam game, hierarki kelas diturunkan lebih lanjut. SkillEffect memiliki subclass khusus yang merepresentasikan berbagai jenis efek akibat penggunaan skill, seperti StunEffect dan WeakenEffect. Demikian pula, ItemEffect dapat memiliki subclass sesuai dengan jenis efek item yang digunakan.

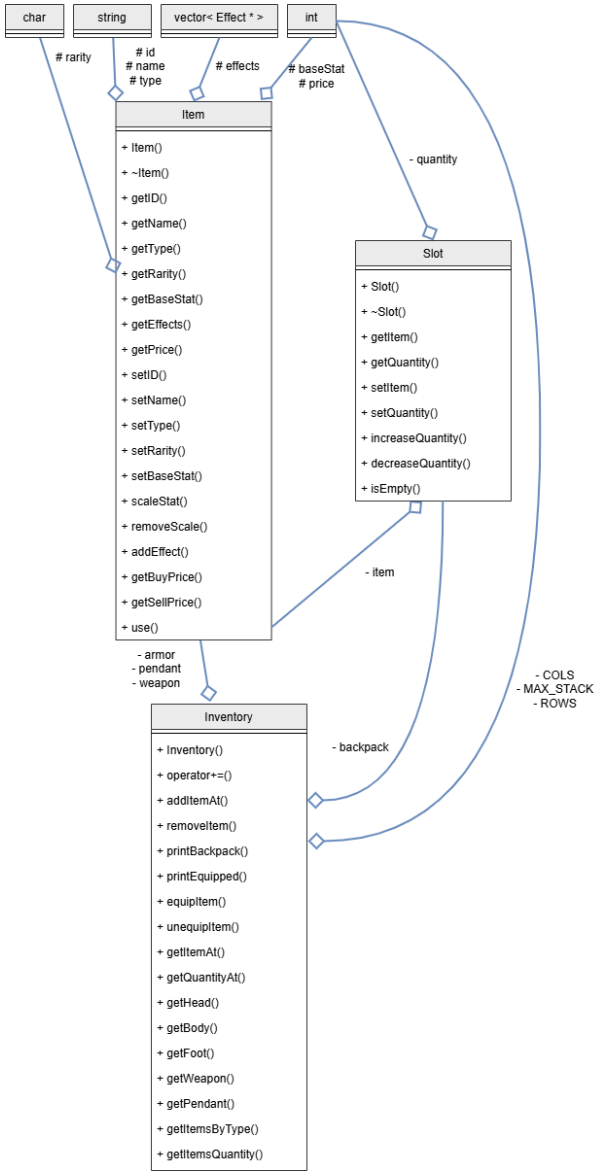
Setiap subclass dapat memiliki atribut dan perilaku tambahan yang lebih spesifik, sesuai dengan peran atau karakteristiknya masing-masing. Desain ini mendukung prinsip DRY (*Don't Repeat Yourself*), sehingga kode menjadi lebih efisien, mudah dikelola, dan dikembangkan.

Kelebihan:

1. Reusability, fungsi-fungsi dasar seperti getDuration() dapat digunakan kembali oleh semua subclass..
2. Fleksibilitas memudahkan pengembangan fitur spesifik masing-masing subclass, seperti atribut tambahan manaLossPerTurn() pada turunan SkillEffect.
3. Struktur yang rapi, memisahkan tanggung jawab kelas secara jelas.

Kekurangan:

1. Keterikatan hierarkis, jika terlalu bergantung pada inheritance, bisa menyulitkan ketika ingin mengubah struktur atau perilaku umum dari kelas Effect.



Desain sistem inventory ini mengimplementasikan agregasi antar kelas utama: Item, Slot, dan Inventory. Kelas Item merepresentasikan entitas dasar dari objek yang bisa digunakan atau disimpan oleh karakter, seperti senjata, armor, atau aksesori. Setiap objek Item memiliki atribut seperti id, name, type, rarity, baseStat, price, serta daftar efek (effects) yang direpresentasikan sebagai vector<Effect*>.

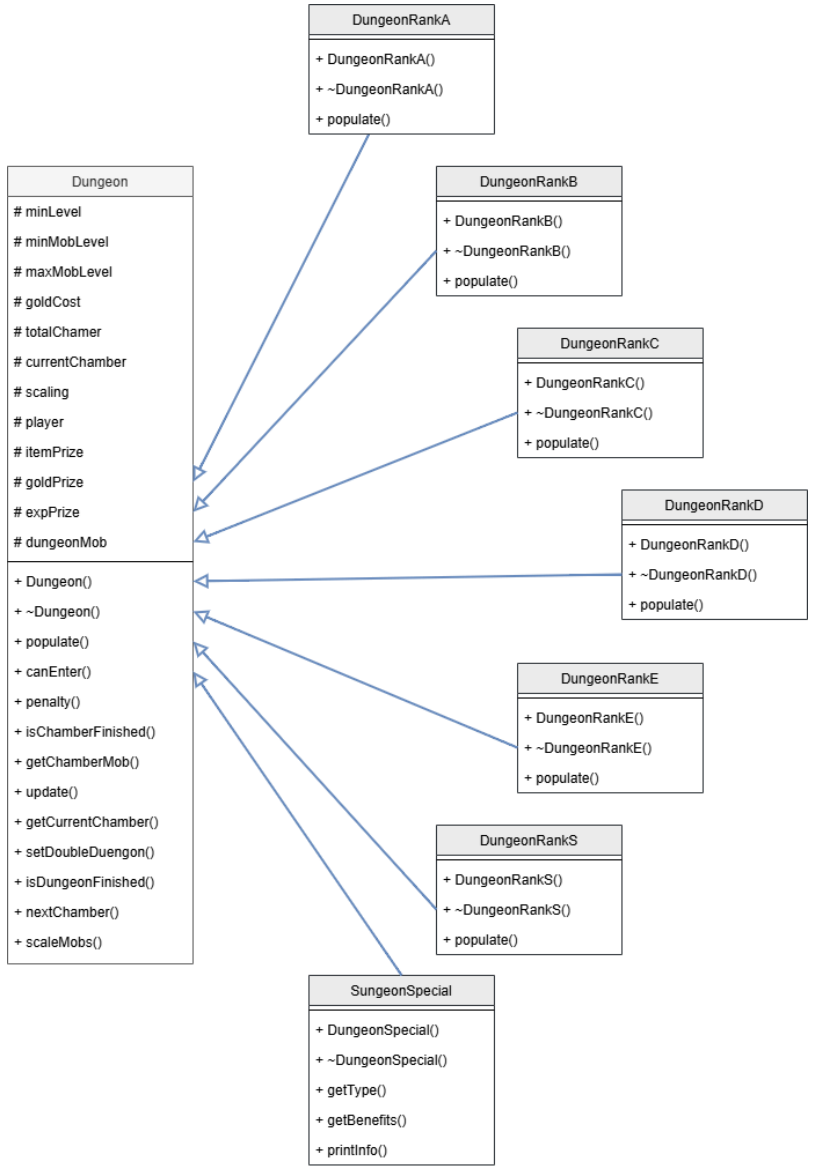
Untuk mengelola banyak item dalam satu tempat, digunakan kelas Slot, yang menyimpan referensi ke Item tertentu serta jumlah (quantity)-nya. Dengan demikian, Slot berperan sebagai wadah untuk satu jenis item dengan kuantitas tertentu.

Selanjutnya, kelas Inventory menjadi pusat dari sistem penyimpanan, yang menyusun banyak Slot ke dalam backpack (representasi array dua dimensi) serta menyediakan slot khusus untuk item yang sedang dipakai (armor, pendant, weapon, dll.). Inventory menyediakan berbagai metode untuk menambah, menghapus, mencetak, dan memanipulasi item dan slot, termasuk pencarian berdasarkan posisi atau tipe.

Kelebihan:

1. Pemanfaatan komposisi dan agregasi, desain memisahkan dengan jelas antara Item, Slot, dan Inventory, serta memanfaatkan agregasi (misalnya Inventory memiliki banyak Slot) untuk menjaga modularitas.
2. Fleksibilitas penggunaan slot, kelas slot memungkinkan pengelolaan jumlah item secara efisien, termasuk operasi peningkatan dan pengurangan jumlah (increaseQuantity(), decreaseQuantity()).
3. Kemudahan eksistensi, sistem Effect yang disimpan dalam Item menggunakan pointer ke kelas Effect, sehingga mudah untuk dikembangkan ke berbagai -tipe efek tanpa mengubah struktur utama item.

	Kekurangan: 1. Ketergantungan pada pointer. Penggunaan vector<Effect*> mengharuskan manajemen memori yang hati-hati agar tidak terjadi kebocoran (*memory leak*).



Pada desain diagram kelas ini, dipilih pendekatan pewarisan (*inheritance*) dengan menjadikan Dungeon sebagai kelas induk (*superclass*) yang memiliki beberapa kelas turunan, yaitu DungeonRankA, DungeonRankB, dan lainnya. Desain ini dipilih karena setiap kelas tersebut memiliki spesifikasi yang berbeda-beda berdasarkan minimal level karakter, total chamber, dan lainnya

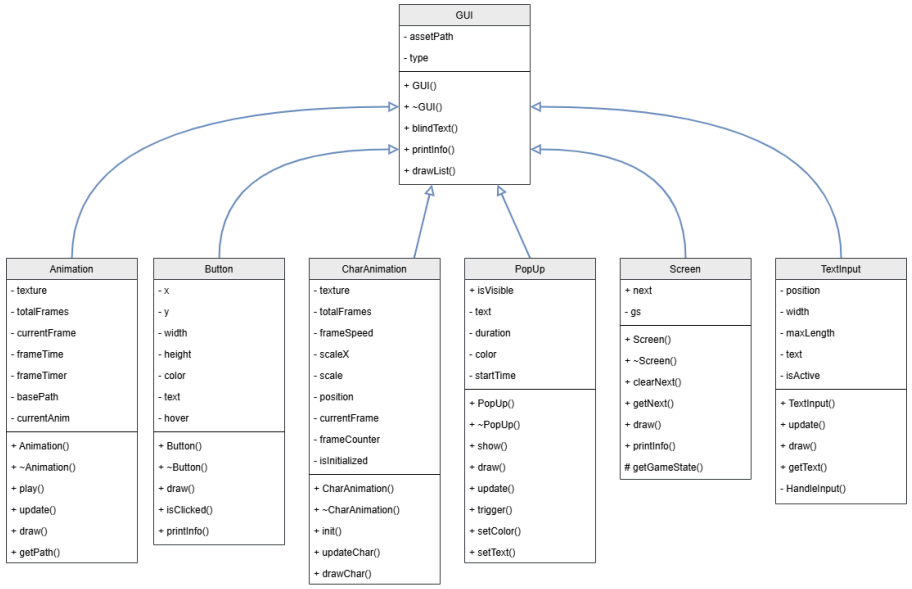
Setiap subclass dapat memiliki atribut dan perilaku tambahan yang lebih spesifik sesuai kebutuhan peran atau karakteristiknya. Desain ini mendukung prinsip DRY (*Don't Repeat Yourself*) sehingga kode lebih mudah dikelola dan dikembangkan.

Kelebihan:

1. Fleksibilitas, memudahkan pengembangan fitur spesifik untuk masing-masing subclass, seperti pengaturan khusus pada method populate(). Akses terhadap fungsionalitas dasar dari kelas induk juga menjadi lebih mudah tanpa perlu mendefinisikan ulang.
2. Struktur yang rapi, memisahkan tanggung jawab kelas secara jelas.

Kekurangan:

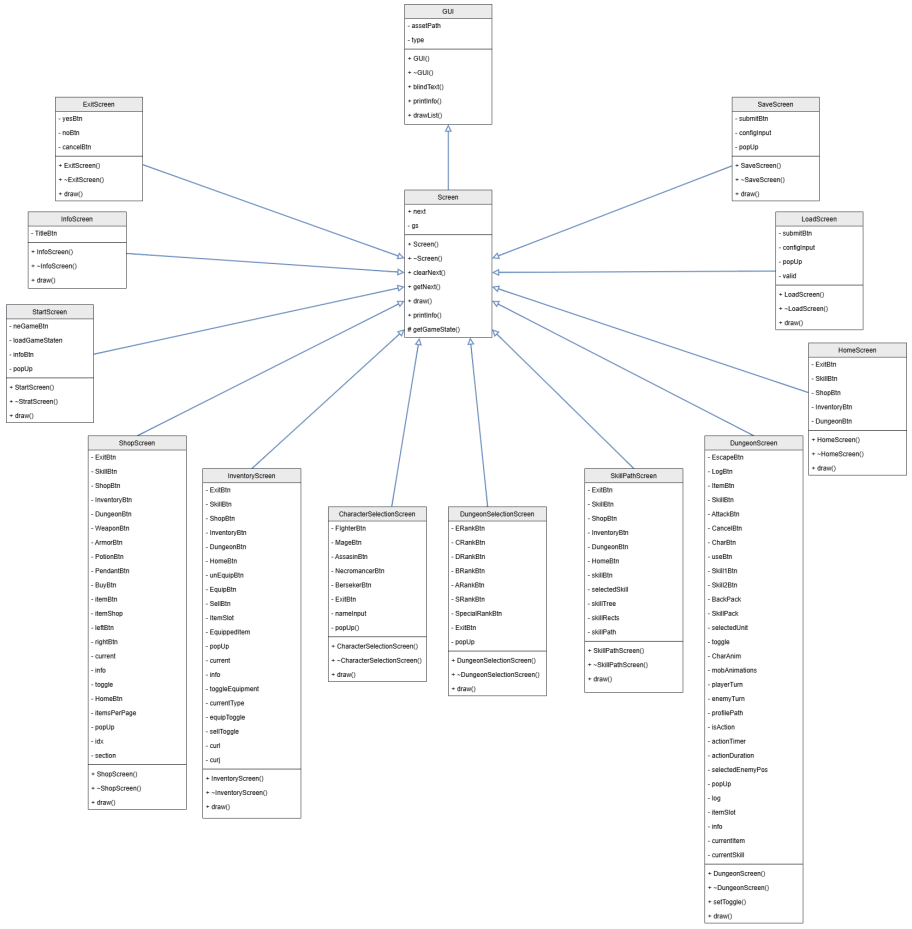
1. Jika perbedaan antar kelas turunan hanya terletak pada implementasi populate(), maka penggunaan inheritance dapat dianggap berlebihan.



Pada desain diagram kelas ini, dipilih pendekatan pewarisan (*inheritance*) dengan menjadikan GUI sebagai kelas induk (superclass) yang memiliki enam kelas turunan, yaitu Animation, Button, CharAnimation, PopUp, Screen, dan TextInput. Desain ini dipilih karena kelas turunan tersebut memiliki atribut dan perilaku dasar yang serupa dan memang termasuk dalam pengaturan untuk GUI.

Kelebihan:

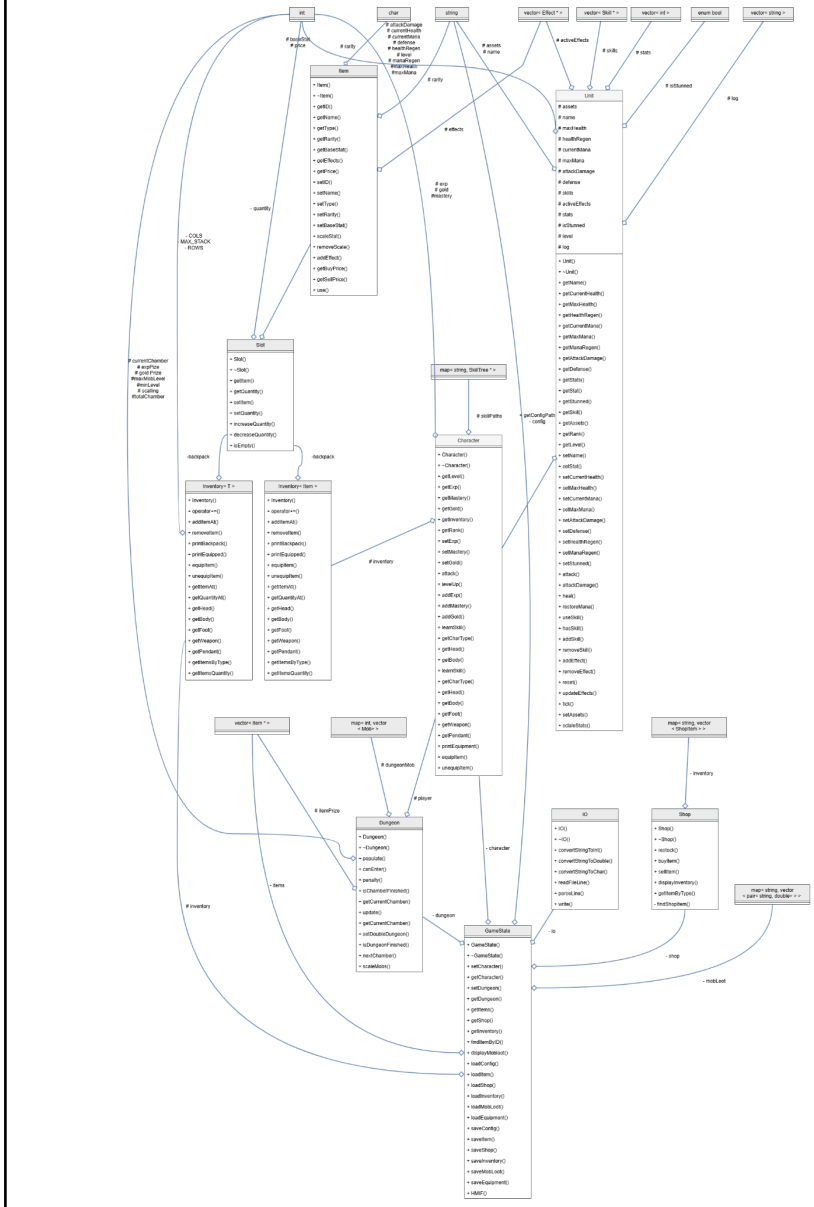
1. Modularitas, bahwa semua fitur GUI tersusun rapi di bawah satu struktur GUI, memudahkan pemeliharaan dan pengembangan.



Pada desain diagram kelas ini, dipilih pendekatan pewarisan (*inheritance*) dengan menjadikan Screen sebagai kelas induk (superclass) yang memiliki banyak turunan yaitu fitur yang beragam untuk menampilkan bagian-bagian yang ada di screen, seperti HomeScreen, InfoScreen, dan lainnya..

Setiap subclass dapat memiliki atribut dan perilaku tambahan yang lebih spesifik sesuai kebutuhan peran atau karakteristiknya. Desain ini mendukung prinsip DRY (*Don't Repeat Yourself*) sehingga kode lebih mudah dikelola dan dikembangkan.
Kelebihan:

1. Fleksibilitas, memudahkan pengembangan fitur spesifik masing-masing subclass, seperti pengaturan tampilan untuk HomeScreen dan DungeonScreen.
2. Struktur yang rapi, memisahkan tanggung jawab kelas secara jelas.



Kelas GameState berperan sebagai pusat kendali utama dalam arsitektur game. Kelas ini bertugas mengelola seluruh aspek penting dari status permainan yang sedang berlangsung, termasuk informasi karakter, dungeon, inventori, dan interaksi dengan shop. Desain ini mengadopsi prinsip komposisi (*composition*) dengan menyatukan berbagai entitas penting ke dalam satu kelas koordinatif

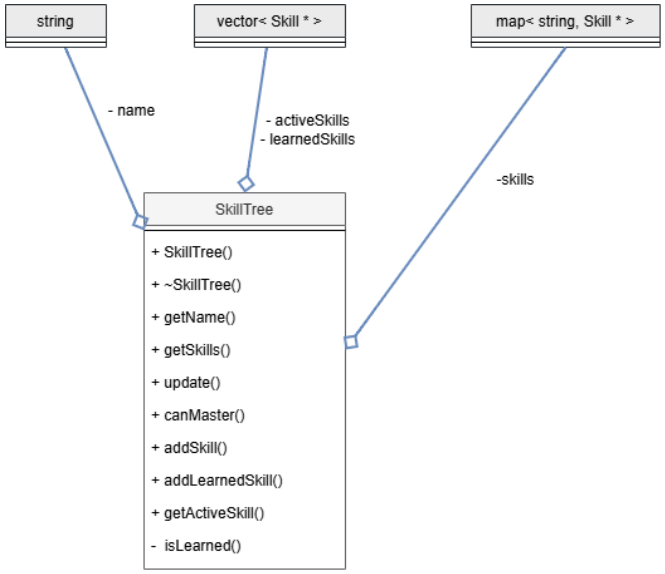
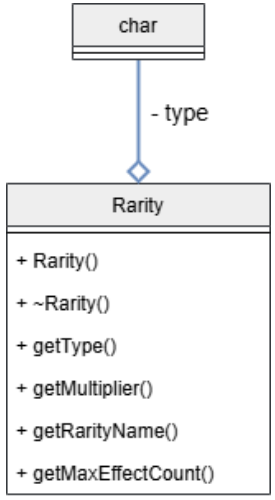
Setiap subclass dapat memiliki atribut dan perilaku tambahan yang lebih spesifik sesuai kebutuhan peran atau karakteristiknya. Desain ini mendukung prinsip DRY (*Don't Repeat Yourself*) sehingga kode lebih mudah dikelola dan dikembangkan.

Kelebihan:

1. Terpusat dan Terstruktur: Semua elemen penting game dikumpulkan dalam satu kelas, memudahkan pengendalian dan pengelolaan status permainan.
2. Modular: Setiap bagian (karakter, dungeon, toko) dikelola oleh kelas tersendiri, memungkinkan pemeliharaan dan pengembangan secara terpisah tanpa memengaruhi GameState secara langsung.
3. Mendukung Ekspansi: Desain ini memungkinkan penambahan fitur seperti sistem simpan-muat game, logika pertarungan, dan transaksi toko tanpa perlu mengubah struktur dasar kelas secara signifikan.

Kekurangan:

1. Ketergantungan tinggi, hal ini dikarenakan GameState mengelola banyak aspek sekaligus, perubahan pada salah satu komponen (misalnya struktur Dungeon atau Character) bisa memicu keharusan untuk menyesuaikan GameState.

<table>
<tr>
<td> 
</td>
<td>Kelas SkillTree didesain menggunakan agregasi terhadap kelas Skill, yang berarti SkillTree menyimpan referensi ke satu atau lebih objek Skill tanpa memiliki kepemilikan penuh atas siklus hidup objek tersebut. Desain ini memungkinkan SkillTree untuk merujuk dan mengelola berbagai skill yang dapat dikonfigurasi atau dipilih oleh karakter dalam game, tanpa secara langsung menciptakan atau menghancurkan objek Skill tersebut.
Kelebihan:
1. Fleksibilitas. Objek Skill dapat digunakan kembali di berbagai SkillTree tanpa duplikasi.
2. Penggunaan memori yang efisien.
3. Modularitas tinggi karena perubahan pada Skill tidak berpengaruh besar pada SkillTree.</td>
</tr>
<tr>
<td> 
</td>
<td>Kelas Rarity didesain dengan nantinya merupakan agregasi dengan kelas lainnya, seperti Item. Kelas ini diputuskan untuk menjadi 1 kelas saja karena hanya mengimplementasikan perbedaan multiplier saja.</td>
</tr>
</table>

2. Penerapan Konsep OOP

2.1. Inheritance & Polymorphism

File: main.cpp

```
// Inisialisasi screen awal
Screen *screen = new StartScreen(gs);
bool RUN = true;
```

```
// Render
BeginDrawing();
ClearBackground(BLACK);
screen->Draw();
EndDrawing();
```

Penjelasan:
Kegunaan konsep *polymorphism* pada main adalah untuk menginisialisasi objek StartScreen dengan menggunakan referensi bertipe Screen. Ini merupakan penerapan dari *polymorphism*, di mana objek dari subclass (StartScreen) diperlakukan sebagai objek dari superclass-nya (Screen). Dengan demikian, method-method yang didefinisikan secara umum di Screen dapat dipanggil, tetapi perilaku spesifik akan mengikuti implementasi milik StartScreen.
Kelebihan:

1. Fleksibilitas dalam pengelolaan objek. Program dapat menangani berbagai jenis Screen secara seragam melalui reference ke Screen.

File: Mob.hpp

```
class Mob : public Unit
{
protected:
    int expReward;
    int goldReward;
    vector<pair<Item *, float>> lootItemProbability;
```

Penjelasan:
Kegunaan konsep *inheritance* pada kelas Mob adalah untuk mewarisi atribut dan method dari superclass Unit. Dengan pewarisan ini, Mob secara otomatis memiliki properti dan fungsi dasar seperti name, currentHealth, maxHealth, attackDamage, dan method-method lainnya yang umum dimiliki oleh seluruh entitas bertipe unit. Hal ini memungkinkan Mob untuk menggunakan kembali kode yang sudah ada tanpa harus menuliskannya ulang.
Kelebihan:
1. Reusability. Atribut dan method umum tidak perlu didefinisikan ulang di Mob, karena sudah tersedia di Unit.
2. Struktur kode yang lebih rapi dan terorganisir.

2.2. Method/Operator Overloading

File: Skill.cpp

```
bool Skill::operator==(const Skill &other) const
{
    return name == other.name && manaCost == other.manaCost && masteryCost == other.masteryCost && damage == other.damage;
}
```

Penjelasan:
Kegunaan konsep *operator overloading* pada kelas Skill digunakan untuk mengecek apakah dua objek Skill yang dibandingkan dianggap sama atau tidak, dengan menggunakan operator ==. Dengan meng-*overload* operator ==, perbandingan dapat disesuaikan berdasarkan atribut-atribut penting dalam kelas, seperti name, manaCost, dan damage.
Keuntungan:
1. Meningkatkan keterbacaan kode

Penggunaan if (skill1 == skill2) lebih intuitif dan mudah dipahami dibandingkan pemanggilan method seperti skill1.isEqual(skill2).

2. Mengurangi duplikasi logika
   Dengan mendefinisikan satu kali logika perbandingan dalam operator==, kita tidak perlu menulis ulang pengecekan manual di berbagai tempat yang membandingkan dua skill.
3. Lebih konsisten dan aman
   Dengan membungkus logika perbandingan dalam operator==, kita mencegah kesalahan akibat perbandingan manual yang tidak lengkap atau tidak konsisten.

File: Effect.cpp

```
bool Effect::operator==(const Effect &other) const
{
    return this->name == other.name && this->description == other.description && this->duration == other.duration && this->remainingDuration == other.remainingDuration && this->effectChance == other.effectChance;
}
```

Penjelasan:

Kegunaan konsep *operator overloading* pada kelas Effect digunakan untuk mengecek apakah dua objek Effect yang dibandingkan dianggap sama atau tidak, dengan menggunakan operator ==. Dengan meng-*overload* operator ==, perbandingan dapat disesuaikan berdasarkan atribut-atribut penting dalam kelas, seperti name, description, duration, remainingDuration, dan effectChance.

Keuntungan:

1. Meningkatkan keterbacaan kode
   Penggunaan if (effect1 == effect2) lebih intuitif dan mudah dipahami dibandingkan pemanggilan method seperti effect1.isEqual(effect2).
2. Mengurangi duplikasi logika
   Dengan mendefinisikan satu kali logika perbandingan dalam operator==, kita tidak perlu menulis ulang pengecekan manual di berbagai tempat yang membandingkan dua skill.
3. Lebih konsisten dan aman
   Dengan membungkus logika perbandingan dalam operator==, kita mencegah kesalahan akibat perbandingan manual yang tidak lengkap atau tidak konsisten.

File: Inventory.cpp.

```cpp
Inventory &Inventory::operator+=(Item *item)
{
    for (int i = 0; i < ROWS; ++i)
    {
        for (int j = 0; j < COLS; ++j)
        {
            Slot &slot = backpack[i][j];
            if (slot.getItem() && slot.getItem()->getID() == item->getID() && slot.getQuantity() < MAX_STACK)
            {
                slot.increaseQuantity(1);
                return *this;
            }
        }
    }

    for (int i = 0; i < ROWS; ++i)
    {
        for (int j = 0; j < COLS; ++j)
        {
            Slot &slot = backpack[i][j];
            if (slot.isEmpty())
            {
                slot.setItem(item);
                slot.setQuantity(1);
                return *this;
            }
        }
    }

    throw string("Backpack penuh! Tidak bisa menambah item.");
}
```

Penjelasan:

Kegunaan konsep operator overloading pada kelas Inventory digunakan untuk menambahkan item ke dalam inventory menggunakan sintaks +=. Dengan meng-*overload* operator +=, proses penambahan item menjadi lebih sederhana dan intuitif, tanpa harus memanggil method seperti addItem() secara eksplisit.

Keuntungan:

1. Sintaks lebih ringkas dan ekspresif.
   Operator += secara semantik sudah menyiratkan aksi penambahan, membuat kode lebih mudah dipahami dan digunakan.

2.3. Template & Generic Classes

File: Necromancer.hpp

```
template <typename T>
int getCountSummonFrom(Unit *unit)
{
    if (T *derived = dynamic_cast<T *>(unit))
    {
        return derived->summon;
    }
    return 0;
}
```

Penjelasan:
Kegunaan konsep template dan generic class pada kelas Necromancer adalah untuk memungkinkan akses terhadap pemanggilan *summon* yang merupakan atribut khusus milik objek Necromancer. Hal ini diperlukan karena di dalam main.cpp, objek karakter umumnya diakses melalui pointer ke kelas induk Unit, sehingga atribut spesifik seperti summon milik Necromancer tidak dapat langsung diakses. Dengan menggunakan *template* atau pendekatan *generic class*, dapat diakses fitur spesifik milik subclass seperti summon secara aman dan fleksibel.
Kelebihan:

1. Dengan template, dapat dipastikan bahwa hanya objek dari tipe yang sesuai seperti Necromancer yang bisa mengakses atribut khusus seperti summon, tanpa harus melakukan casting manual.

2.4. Exception

File: Inventory.cpp

```cpp
bool Inventory::addItemAt(Item *item, int i, int j)
{

    if (i < 0 || i >= ROWS || j < 0 || j >= COLS)
    {
        throw std::out_of_range("Indeks slot tidak valid.");
    }

    Slot &slot = backpack[i][j];

    if (slot.getItem() && slot.getItem()->getID() == item->getID() && slot.getQuantity() < MAX_STACK)
    {
        slot.increaseQuantity(1);
        return true;
    }

    else if (slot.isEmpty())
    {
        slot.setItem(item);
        slot.setQuantity(1);
        return true;
    }

    throw std::runtime_error("Slot tidak dapat menampung item.");
}
```

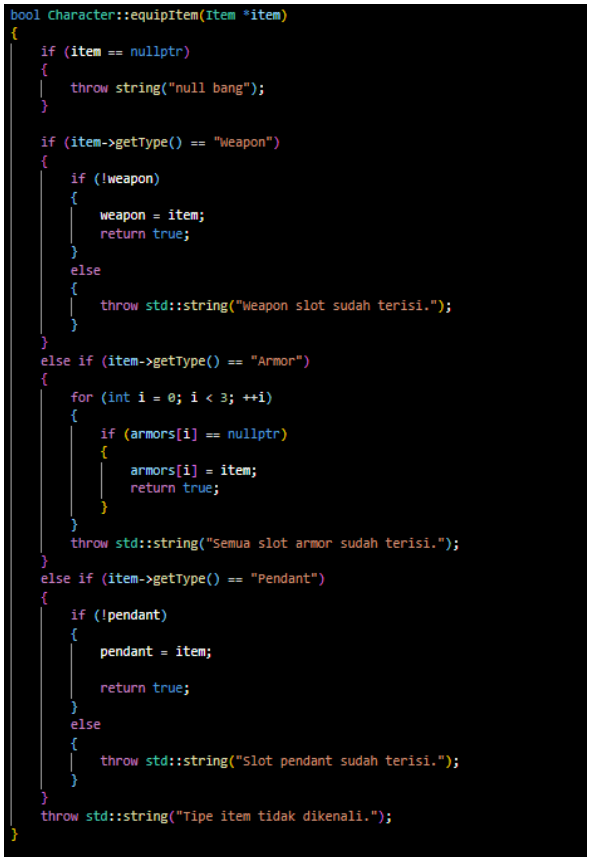
Penjelasan:

Kegunaan konsep *exception* pada kelas Inventory adalah untuk menangani kondisi-kondisi yang tidak diinginkan, seperti saat pengguna mencoba mengakses indeks yang tidak valid atau menambahkan item ke slot yang sudah penuh. Dengan menerapkan exception, alur program tetap dapat dikendalikan secara aman tanpa menyebabkan crash atau hasil yang tidak terduga.

Kelebihan:

1. Meningkatkan stabilitas program, exception mencegah aplikasi berhenti tiba-tiba karena kesalahan akses, dengan memberikan pesan kesalahan yang dapat ditangani dengan baik oleh program.
2. Mempermudah debugging dan pelacakan kesalahan. Exception dapat menjadi penanda saat ada kesalahan akses yang dilakukan.
3. Mendukung validasi input secara terstruktur. Dengan exception, dapat langsung menolak operasi yang tidak valid, seperti pengaksesan yang tidak valid.

File: Character.cpp



```cpp
bool Character::equipItem(Item *item)
{
    if (item == nullptr)
    {
        throw string("null bang");
    }

    if (item->getType() == "Weapon")
    {
        if (!weapon)
        {
            weapon = item;
            return true;
        }
        else
        {
            throw std::string("Weapon slot sudah terisi.");
        }
    }
    else if (item->getType() == "Armor")
    {
        for (int i = 0; i < 3; ++i)
        {
            if (armors[i] == nullptr)
            {
                armors[i] = item;
                return true;
            }
        }
        throw std::string("Semua slot armor sudah terisi.");
    }
    else if (item->getType() == "Pendant")
    {
        if (!pendant)
        {
            pendant = item;

            return true;
        }
        else
        {
            throw std::string("Slot pendant sudah terisi.");
        }
    }
    throw std::string("Tipe item tidak dikenali.");
}
```

Penjelasan:

Kegunaan konsep *exception* pada kelas Character di method equipItem() adalah untuk menangani kondisi-kondisi yang tidak diinginkan, seperti ketika pemain mencoba menggunakan item yang tidak sesuai dengan tipe yang diperbolehkan atau ketika item tersebut sudah dalam kondisi terpakai. Dengan menerapkan exception, program tetap dapat berjalan secara aman dan terkendali tanpa mengalami *crash* atau menghasilkan perilaku yang tidak diinginkan.

Kelebihan:

1. Exception memungkinkan penolakan langsung terhadap aksi tidak valid, seperti mencoba memakai item dengan tipe yang tidak cocok atau jika slot item sudah terisi.

File: Shop.cpp

```cpp
bool Shop::buyItem(const std::string &id)
{
    try
    {
        ShopItem *shopItem = findShopItem(id);
        if (!shopItem)
        {
            throw std::invalid_argument("Item dengan ID '" + id + "' tidak ditemukan di shop.");
        }

        if (!shopItem->decreaseQuantity(1))
        {
            throw std::out_of_range("Stok item '" + shopItem->getItem()->getName() + "' habis.");
        }

        std::cout << "Item " << shopItem->getItem()->getName() << " dibeli.\n";
        return true;
    }
    catch (const std::exception &e)
    {
        std::cerr << "Gagal membeli item: " << e.what() << std::endl;
        return false;
    }
}
```

Penjelasan:

Kegunaan konsep *exception* pada kelas Shop di method buyItem() adalah untuk menangani kondisi-kondisi yang tidak diinginkan, seperti ketika pemain mencoba membeli item yang tidak tersedia atau habis. Dengan menerapkan exception, program tetap dapat berjalan secara aman dan terkendali tanpa mengalami *crash* atau menghasilkan perilaku yang tidak diinginkan.

Kelebihan:

1. Exception memungkinkan penolakan langsung terhadap aksi tidak valid, seperti mencoba membeli item yang tidak valid.
2. Pengendalian alur program yang aman. Kesalahan dapat ditangani melalui blok try-catch tanpa menghentikan eksekusi program secara tiba-tiba.

2.5. C++ Standard Template Library

#include<vector>

File: Shop.cpp
`std::vector<ShopItem> Shop::getItemsByType(const std::string &type) const`
`{`
`    auto it = inventory.find(type);`
`    if (it != inventory.end())`
`    {`
`        return it->second;`
`    }`
`    return {};`
`}`

Penjelasan: Fungsi getItemsByType digunakan untuk mengambil daftar item dari inventory toko berdasarkan tipe seperti "Weapon" atau "Armor". Fungsi ini mencari apakah tipe yang diminta terdapat dalam struktur inventory. Jika ditemukan, akan mengembalikan **vektor** ShopItem yang sesuai dengan tipe tersebut. Jika tidak ditemukan, fungsi akan mengembalikan vektor kosong. Fungsi ini memudahkan akses terhadap item-item toko yang dikategorikan berdasarkan type nya.
#include <map>
File: Shop.cpp

```cpp
void Shop::restock(const std::vector<Item *> &items)
{
    for (Item *item : items)
    {
        int quantityPotion;
        std::string type = item->getType();
        if (type == "Potion")
        {
            quantityPotion = 5;
        }
        else
        {
            quantityPotion = 1;
        }
        bool added = false;

        for (auto &shopItem : inventory[type])
        {
            if (shopItem.getItem()->getID() == item->getID())
            {
                shopItem.increaseQuantity(quantityPotion);
                added = true;
                break;
            }
        }

        if (!added)
        {
            inventory[type].emplace_back(item, quantityPotion);
        }
    }
```

Penjelasan:

Fungsi restock digunakan untuk menambahkan atau memperbarui stok item pada toko berdasarkan daftar item yang diberikan. Fungsi ini bekerja dengan memanfaatkan struktur data **map** bernama inventory, yang memetakan string dan tipe item vector<ShopItem>. Untuk setiap item yang akan di-restock, fungsi akan mengecek tipenya, lalu menentukan jumlah stok default . Selanjutnya, fungsi mencari apakah item dengan ID yang sama sudah ada dalam vektor ShopItem yang sesuai di dalam map. Jika ditemukan, kuantitasnya ditambah. jika tidak ditemukan, item baru ditambahkan ke dalam vektor tersebut menggunakan emplace_back. Dengan demikian, fungsi ini menjaga agar stok

setiap tipe item dalam map inventory tetap terorganisir dan diperbarui sesuai kebutuhan.

#include <filesystem>

File: LoadScreen.cpp

```
if (submitBtn.isClicked())
{
    try
    {
        string path = string("./data/") + configInput.GetText();
        if (std::filesystem::exists(path) && std::filesystem::is_directory(path))
        {

            getGameState().loadConfig(path);
            getGameState().setConfigPath(configInput.GetText());
            valid = true;
            popUp.trigger("Horay, Load success !", GREEN, 0.8);
            popUp.show();
        }
        else
        {
            throw string("Folder not found...");
        }
    }
    catch (string &e)
    {
        popUp.trigger(e, RED, 0.8);
        popUp.show();
    }
}
```

Penjelasan:

Fungsi pada gambar ini memanfaatkan fitur **filesystem** untuk memvalidasi keberadaan dan jenis path (folder) yang dimasukkan oleh pengguna sebelum memuat konfigurasi game. Ketika tombol submitBtn di klik, program akan mengambil teks dari configInput, lalu menggabungkannya dengan path "./data/" untuk membentuk path lengkap. Dengan menggunakan kode “std::filesystem::exists(path) && std::filesystem::is_directory(path)”, fungsi memastikan bahwa path tersebut benar-benar ada dan merupakan sebuah directory. Jika valid, maka konfigurasi game akan dimuat dan status sukses akan ditampilkan. Jika folder tidak ditemukan,maka akan dilempar exception yang ditangani oleh blok catch, dan pesan kesalahan ditampilkan dengan popup berwarna merah. Dengan demikian, filesystem di sini berperan dalam memastikan keamanan dan validitas path sebelum data game diakses.

#include <cmath>

File: GUI.cpp

```
void GUI::blinkText(string s, int x, int y, int font)
{
    if (fmod(GetTime(), 1.0) < 0.5)
    {
        DrawText(s.c_str(), x, y, font, WHITE);
    }
}
```

Penjelasan:
Fungsi blinkText pada gambar menggunakan fungsi fmod dari library **cmath**. Fungsi fmod(GetTime(), 1.0) digunakan untuk menghasilkan nilai pecahan waktu saat ini dalam satuan detik (misalnya 0.0 sampai <1.0), yang memungkinkan teks blink dengan interval sekitar 0.5 detik. Jika nilai pecahan waktu tersebut kurang dari 0.5, maka teks akan ditampilkan menggunakan DrawText.

#include <algorithm>

File: Unit.cpp

```
void Unit::removeEffect(Effect *effect)
{
    auto it = find(activeEffects.begin(), activeEffects.end(), effect);
    if (it != activeEffects.end())
    {
        (*it)->remove(this);
        delete *it;
        activeEffects.erase(it);
    }
}
```

Penjelasan:
Fungsi removeEffect pada gambar menggunakan fungsi find yang merupakan bagian dari library algorithm. Fungsi find digunakan untuk mencari pointer effect dalam container activeEffects yang kemungkinan merupakan vector<Effect*>. Jika ditemukan, efek tersebut akan dihapus dari unit dengan memanggil remove(this), lalu dihapus dari memori menggunakan delete, dan akhirnya dihapus dari container dengan erase.

2.6. Konsep OOP lain

2.6.1 Aggregation

File: ShopItem.hpp

```
class ShopItem {
private:
    Item* item;
    int quantity;
public:
    ShopItem(Item* item, int quantity);
    Item* getItem() const;
    int getQuantity() const;
    void increaseQuantity(int amount);
    bool decreaseQuantity(int amount);
};
```

Penjelasan:
Kelas ShopItem pada gambar menerapkan konsep agregasi melalui penggunaan pointer Item* item, yang menunjukkan bahwa objek ShopItem memiliki referensi ke objek Item, namun tidak memiliki siklus hidup penuh terhadapnya. Artinya, ShopItem tidak bertanggung

jawab atas pembuatan atau penghancuran objek Item, melainkan hanya mengakses atau menggunakan objek tersebut selama diperlukan. Hal ini mencerminkan hubungan "has-a" yang longgar, di mana ShopItem bergantung pada keberadaan Item tanpa mengelolanya secara penuh, yang merupakan ciri khas dari agregasi dalam paradigma pemrograman berorientasi objek.

File: Character.hpp

```
class Character : public Unit
{
protected:
    int exp;
    int mastery;
    int gold;
    SkillTree *skillPath;
```

Penjelasan:
Pada kelas Character, konsep agregasi diterapkan melalui pointer SkillTree* skillPath, yang menunjukkan bahwa objek Character memiliki referensi ke objek SkillTree tanpa memiliki kepemilikan penuh terhadap siklus hidupnya. Artinya, Character menggunakan objek SkillTree sebagai bagian dari fungsionalitasnya (misalnya untuk mengakses pohon skill), namun tidak bertanggung jawab untuk membuat atau menghancurkannya. Ini mencerminkan hubungan "has-a" yang longgar antara Character dan SkillTree, di mana SkillTree bisa juga digunakan atau dimiliki oleh objek lain di luar Character, sesuai prinsip agregasi dalam OOP.

File: Item.hpp

```
class Item
{
protected:
    std::string id;
    std::string name;
    std::string type;
    char rarity;
    int baseStat;
    vector<Effect *> effects;
    int price;
```

Penjelasan:
Kelas Item pada gambar menunjukkan penerapan **agregasi** melalui atribut vector<Effect*> effects, yang berarti setiap objek Item dapat memiliki beberapa efek (Effect) yang terkait dengannya tanpa memiliki kendali penuh atas siklus hidup objek Effect tersebut. Dengan menyimpan pointer ke objek Effect, kelas Item hanya merujuk pada efek yang mungkin juga digunakan atau dihapus oleh entitas lain di sistem. Ini mencerminkan hubungan "has-a" yang longgar antara Item dan Effect, di mana Item menggunakan efek untuk memperkuat fungsionalitasnya, namun tidak bertanggung jawab atas penciptaan atau penghancuran efek tersebut sesuai prinsip agregasi dalam pemrograman berorientasi objek.

2.6.2 Abstract Base Class

File: Character.hpp

```
class Character : public Unit
{
protected:
    int exp;
    int mastery;
    int gold;
    SkillTree *skillPath;


    Inventory *inventory;

    // item
    Item *armors[3];
    Item *pendant = nullptr;
    Item *weapon = nullptr;

public:
    virtual void levelUp() = 0;
    virtual void initPath() = 0;
    virtual string getCharType() const = 0;
```

Penjelasan:
Penggunaan abstract base class pada class Character dimaksudkan karena setiap jenis karakter dalam game, seperti Fighter, Mage, Assassin, Necromancer, dan Berseker, memiliki metode yang perilakunya berbeda dan harus diimplementasikan secara spesifik di kelas turunannya. Class Character mendefinisikan beberapa *pure virtual function* seperti levelUp(), initPath(), dan getCharType(), yang menandakan bahwa setiap subclass wajib menyediakan implementasi konkret untuk fungsi-fungsi tersebut. Hal ini penting karena proses kenaikan level, jalur pengembangan skill, dan penentuan tipe karakter memiliki logika yang berbeda untuk setiap jenis karakter. Dengan menjadikan Character sebagai abstract base class, program mencegah pembuatan objek langsung dari Character, dan hanya mengizinkan instansiasi dari kelas turunan yang telah mengimplementasikan metode-metode tersebut.

File: Effect.hpp

```
class Effect
{
protected:
    string name;
    string description;
    int duration;
    int remainingDuration;
    float effectChance;

public:
    virtual void apply(Unit *target) = 0;
    virtual void remove(Unit *target) = 0;
    virtual Effect *clone() const = 0;
```

Penjelasan:
Penggunaan abstract base class pada class Effect dimaksudkan karena setiap jenis efek dalam game, seperti *Burn*, *Stun* dan lainnya, memiliki perilaku yang berbeda dalam hal penerapan efek (apply), penghapusan efek (remove), dan penyalinan efek (clone). Ketiga fungsi tersebut dideklarasikan sebagai *pure virtual functions*, yang berarti setiap subclass dari Effect wajib mengimplementasikan fungsi-fungsi tersebut sesuai dengan karakteristik efek masing-masing. Misalnya, efek *Burn* dapat mengurangi HP target setiap giliran, sedangkan *Stun* dapat menghentikan aksi target untuk beberapa giliran. Dengan menjadikan Effect sebagai abstract base class, program tidak mengizinkan pembuatan objek langsung dari Effect, melainkan hanya dari subclass yang sudah mengimplementasikan perilaku spesifik.

File: Mob.hpp

```
class Mob : public Unit
{
protected:
    int expReward;
    int goldReward;
    vector<pair<Item *, float>> lootItemProbability;

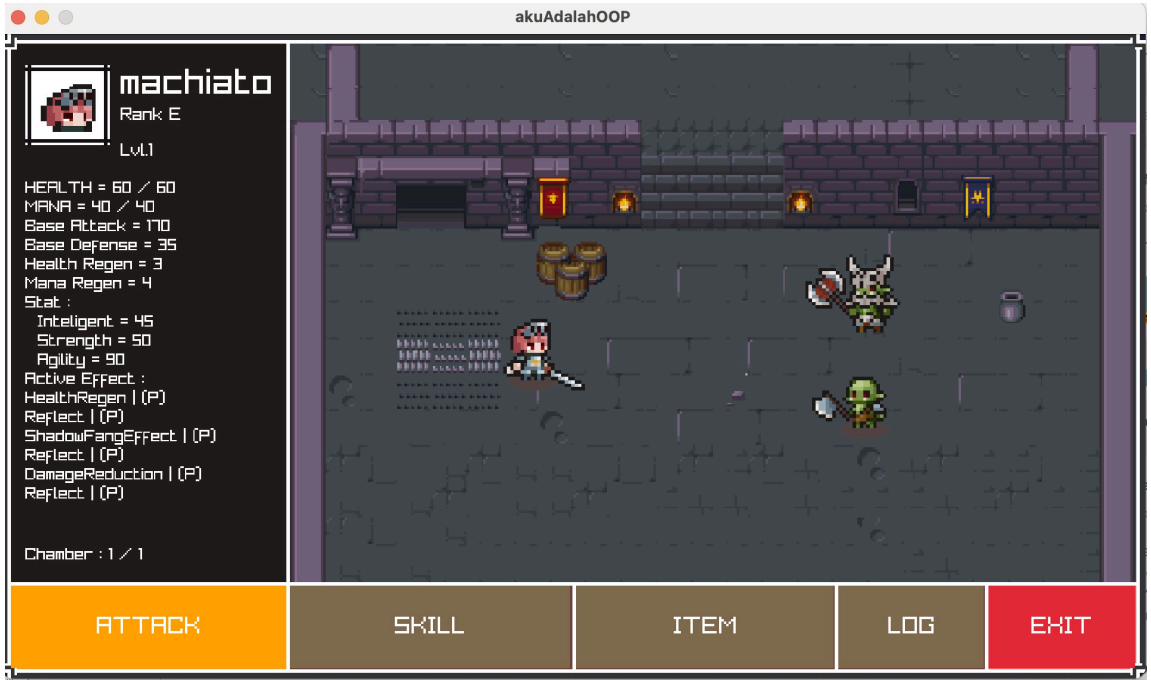
public:
    virtual void act(Unit *target) = 0;
    virtual string getType() = 0;
```

Penjelasan:
Penggunaan abstract base class pada class Mob dimaksudkan untuk menangani berbagai jenis mob dalam game dengan perilaku yang berbeda, seperti BasicMob dan BossMob. Class ini mendefinisikan fungsi act() dan getType() sebagai *pure virtual function*, yang mengharuskan setiap kelas turunan untuk mengimplementasikan logikanya masing-masing. Misalnya, pada BasicMob, fungsi act() hanya berfungsi untuk menambahkan sejumlah heal ke dirinya sendiri, sedangkan, pada BossMob fungsi act() tidak hanya melakukan heal, tetapi juga memicu rage skill ketika HP-nya berada di bawah 20% dari maksimum HP-nya.

3. Bonus Yang dikerjakan

3.1. Bonus yang diusulkan oleh spek

3.1.1. Antarmuka Pengguna Grafis (APG)



Pembuatan antarmuka pengguna grafis menggunakan library raylib. Library ini berbasis c++ dan dapat diimplementasikan secara sederhana pada proyek. Struktur PGA pada proyek ini terpusat dari satu kelas *parrent* utama yaitu GUI. GUI memiliki beberapa *child,* yaitu Animation, Button, PopUp, Screen, TextInput, dan CharAnimation.

3.1.1.1. GUI

```
class GUI
{
private:
    string assetPath;
    string type;

public:
    GUI();
    virtual ~GUI();

    void blinkText(string text, int x, int y, int font);
    void printInfo();
    void drawList(vector<string> list, int x, int y, int fontSize, Color color);
    void drawListCol(const vector<string> &list, int startX, int startY, int itemsP
};
```

GUI merupakan kelas induk (parent class) yang dirancang untuk menjadi dasar bagi berbagai kelas yang berkaitan dengan tampilan visual dalam aplikasi. Method-method dalam GUI berfungsi untuk menampilkan elemen-elemen visual dasar yang dapat digunakan dan dikembangkan lebih lanjut oleh kelas-kelas turunannya.

3.1.1.2. Animation

```
class Animation : public GUI
{
public:
    Animation(const std::string &basePath, float frameTime = 0.1f);
    ~Animation();

    void Play(const std::string &animationName);
    void Update();
    void Draw(Vector2 position, float scale = 1.0f);

    std::string getPath() const;

private:
    Texture2D texture;
    int totalFrames;
    int currentFrame;
    float frameTime;
    float frameTimer;
    std::string basePath;
    std::string currentAnim;
};
```

```
class CharAnimation : public GUI
{
private:
    Texture2D texture;
    int totalFrames;
    int frameSpeed;
    float scaleX;
    float scaleY;
    Vector2 position;
    int currentFrame;
    int frameCounter;
    bool isInitialized;

public:
    CharAnimation();
    ~CharAnimation();

    void Init(const std::string &filePath, int totalFrames, int frameSpeed,
              float scaleX, float scaleY, Vector2 position);

    void UpdateChar();
    void DrawChar();
};
```

Kelas Animation berfungsi untuk menampilkan animasi pada layar, di mana animasi tersebut diambil dari gambar *sprite asset* yang berisi kumpulan *frame* animasi.

3.1.1.3. Button

```
class Button : public GUI
{
private:
    int x, y;
    int width, height;
    Color color;
    const char *text;
    bool hover;
    Sound hoverSfx;
    Sound clickSfx;
    bool wasHovering = false;

public:
    Button(int x, int y, int width, int height);
    Button(int x, int y, int width, int height, const char *text, Color color = DARKGRAY);
    ~Button();
    void draw();
    bool isClicked();
    void printInfo();
};
```

Button berfungsi untuk menampilkan elemen tombol interaktif pada layar. Kelas ini menyimpan informasi seperti koordinat posisi, warna, dan teks yang ditampilkan pada tombol.

3.1.1.4. PopUp

```
class PopUp : GUI
{
private:
    string text;
    double duration;
    Color color;
    double startTime;

public:
    PopUp();
    PopUp(std::string text, double duration, Color color);
    ~PopUp();
    bool isVisible;
    void show();
    void update();
    void draw();

    void trigger(std::string message, Color color, double duration);

    void setColor(Color color);
    void setText(std::string s);
};
```

Kelas PopUp berfungsi untuk menampilkan jendela pop-up pada layar, yang digunakan sebagai notifikasi untuk memberikan informasi penting kepada pengguna, seperti pesan sukses atau peringatan terhadap exception yang terjadi dalam aplikasi.

3.1.1.5. TextInput

```
class TextInput : public GUI
{
public:
    TextInput(Vector2 position, int width, int maxLength);

    void Update();
    void Draw();
    std::string GetText() const;

private:
    Vector2 position;
    int width;
    int maxLength;
    std::string text;
    bool isActive;

    void HandleInput();
};
```

Kelas TextInput berfungsi untuk menerima input teks dari pengguna. Kelas ini menangkap dan menyimpan teks yang diketik oleh pengguna melalui keyboard.

3.1.1.6. Screen

```
class Screen : public GUI
{
private:
    string next;
    GameState &gs;

public:
    Screen(GameState &gs);
    void setNext(string s);
    void clearNext();
    string getNext();
    virtual ~Screen();

    virtual void Draw() = 0;

    void printInfo();

protected:
    GameState &getGameState();
};
```

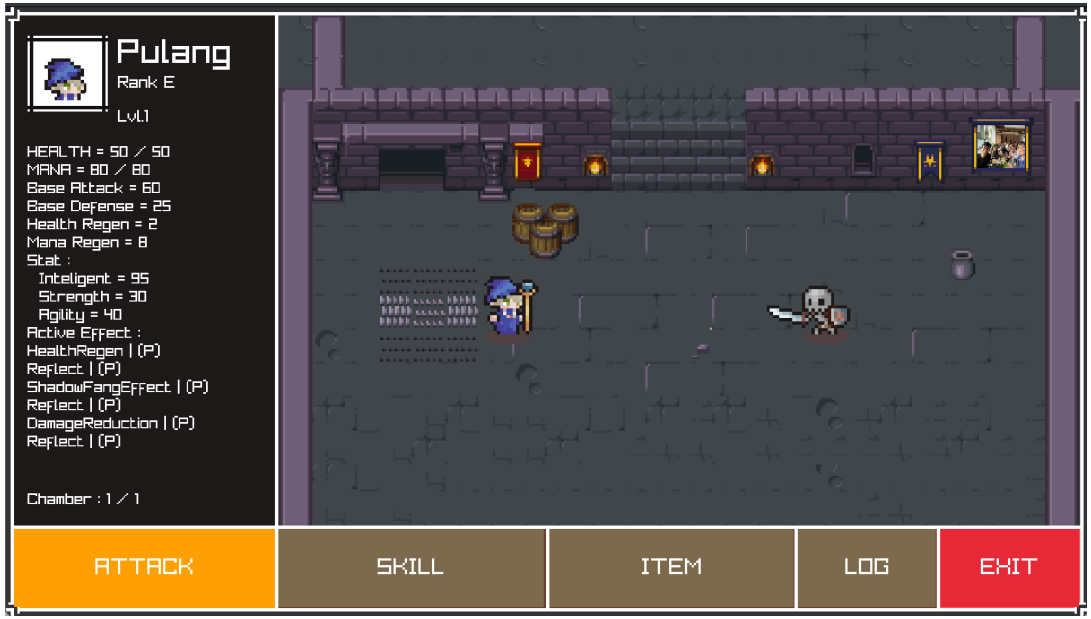
Kelas Screen merupakan kelas utama yang menjadi wadah bagi objek-objek lain untuk berinteraksi. Screen berperan sebagai "dunia" tempat berbagai objek ditampilkan, berperilaku, dan saling berinteraksi selama aplikasi berjalan.

3.1.1.6.1. CharacterSelectionScreen



Kelas CharacterSelectionScreen berfungsi untuk menampilkan menu pemilihan karakter yang akan dimainkan oleh pengguna. Layar ini memungkinkan pemain memilih karakter sebelum memulai permainan

3.1.1.6.2. DungeonScreen



Kelas DungeonScreen merupakan inti dari permainan, di mana pengguna dapat memainkan karakter yang telah dipilih untuk menjelajahi dungeon dan berinteraksi dengan elemen-elemen game.

3.1.1.6.3. DungeonSelectionScreen



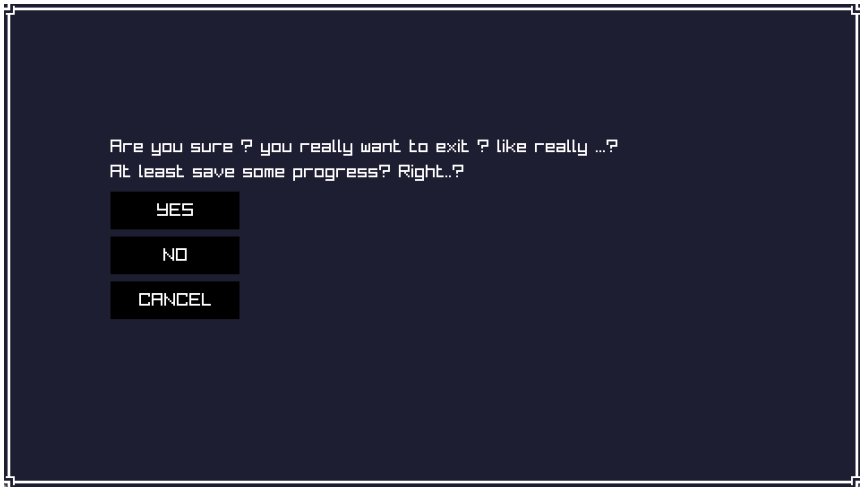
Kelas DungeonSelectionScreen menampilkan daftar pilihan rank dungeon yang dapat dipilih dan dimasuki oleh pemain. Layar ini memungkinkan pengguna memilih tingkat kesulitan dungeon sebelum memulai petualangan.

3.1.1.6.4. EndDungeonScreen



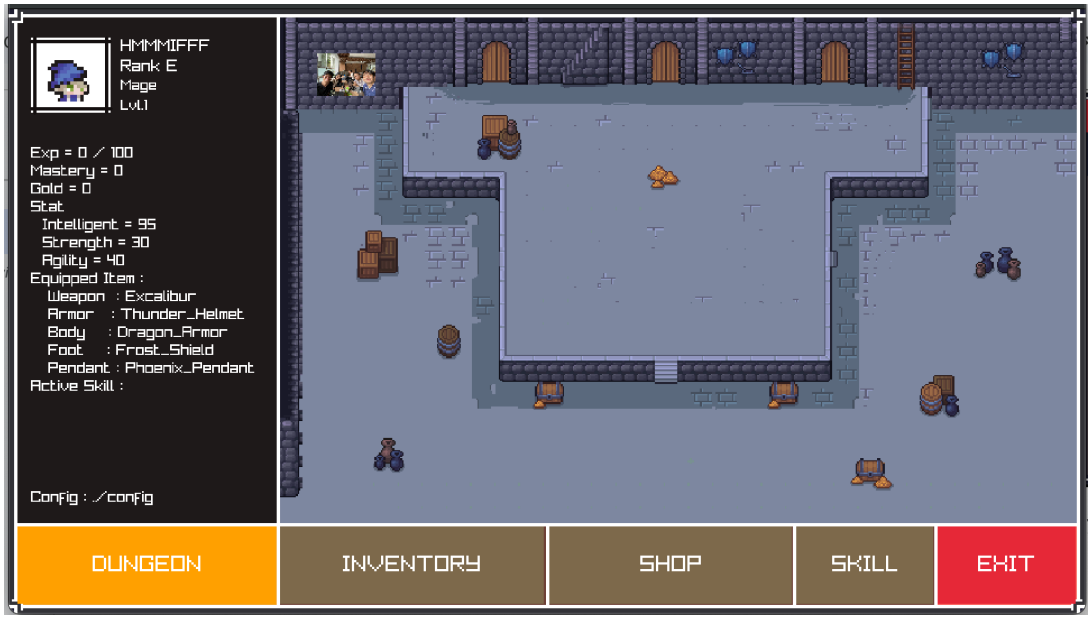
Kelas EndDungeonScreen menampilkan hasil dari perjalanan dungeon yang telah diselesaikan oleh pemain. Layar ini biasanya menampilkan statistik akhir—seperti gold penalty, exp penalty, dan item yang didapat dari Mob—serta opsi untuk kembali ke menu utama.

3.1.1.6.5. ExitScreen



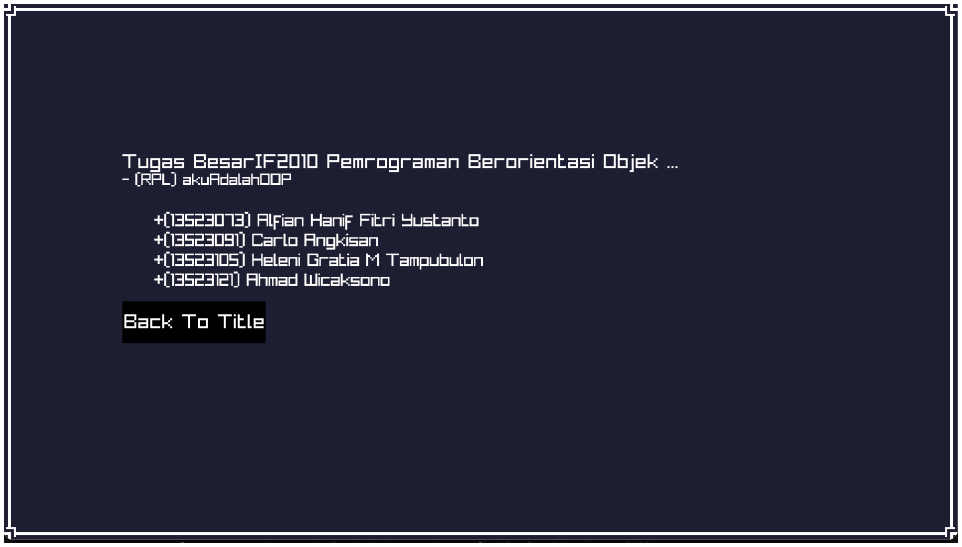
Kelas ExitScreen menampilkan tampilan konfirmasi sebelum pemain keluar dari permainan. Layar ini menyajikan opsi seperti “Ya” untuk menyimpan progress terakhir atau “Tidak” untuk tidak menyimpan. Disediakan juga opsi “Cancel” jika ingin kembali melanjutkan permainan.

3.1.1.6.6. HomeScreen



Kelas HomeScreen berfungsi sebagai menu utama, di mana pemain dapat memilih berbagai fitur dalam permainan—misalnya memulai game, mengakses inventori, akses shop, skill, atau keluar.

3.1.1.6.7. InfoScreen



Kelas InfoScreen menampilkan informasi tentang aplikasi, termasuk nama dan NIM kontributor yang terlibat dalam pengembangan aplikasi ini.

3.1.1.6.8. InventoryScreen



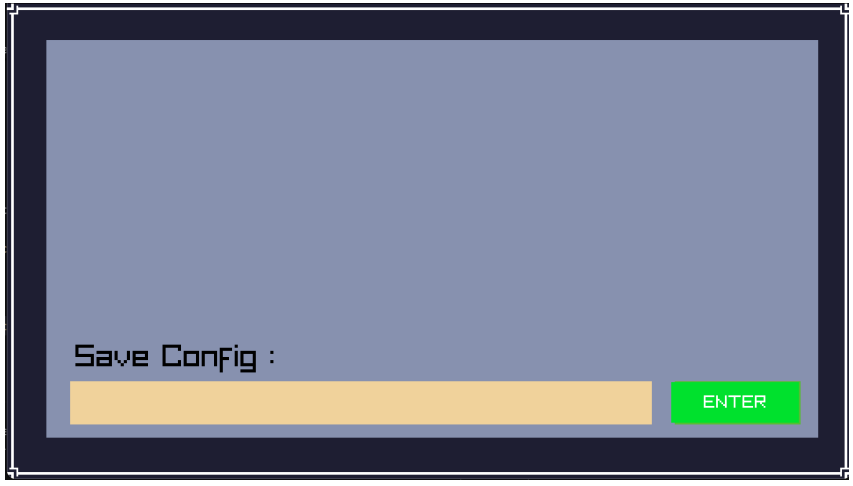
Kelas InventoryScreen berfungsi untuk menampilkan menu inventori milik pengguna. Layar ini memperlihatkan daftar item yang dimiliki, slot–slot yang tersedia, serta menyediakan aksi seperti memakai item (equip) dalam inventori.

3.1.1.6.9. LoadScreen



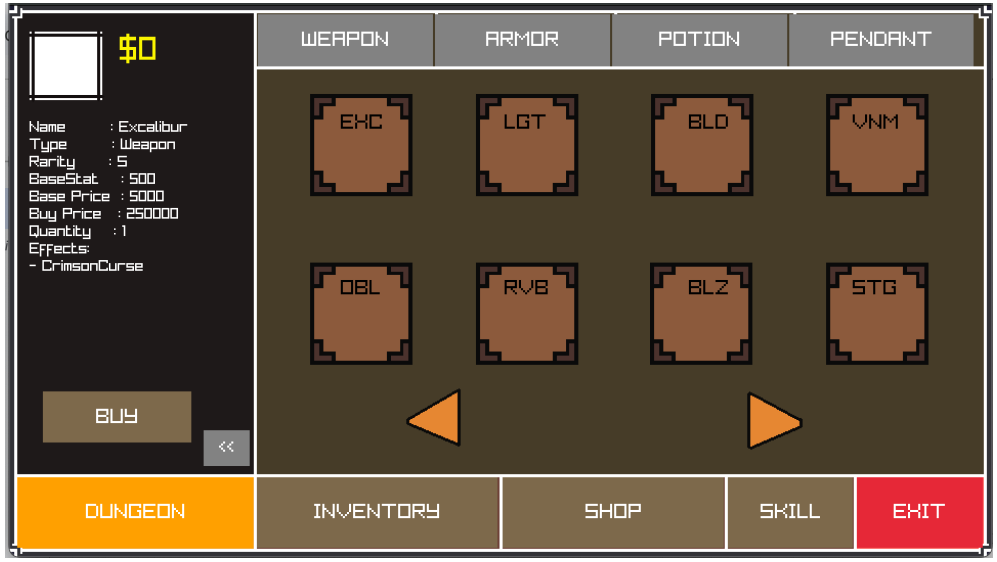
Kelas LoadScreen berfungsi sebagai antarmuka untuk memuat (load) file konfigurasi dari folder config.

3.1.1.6.10. SaveScreen



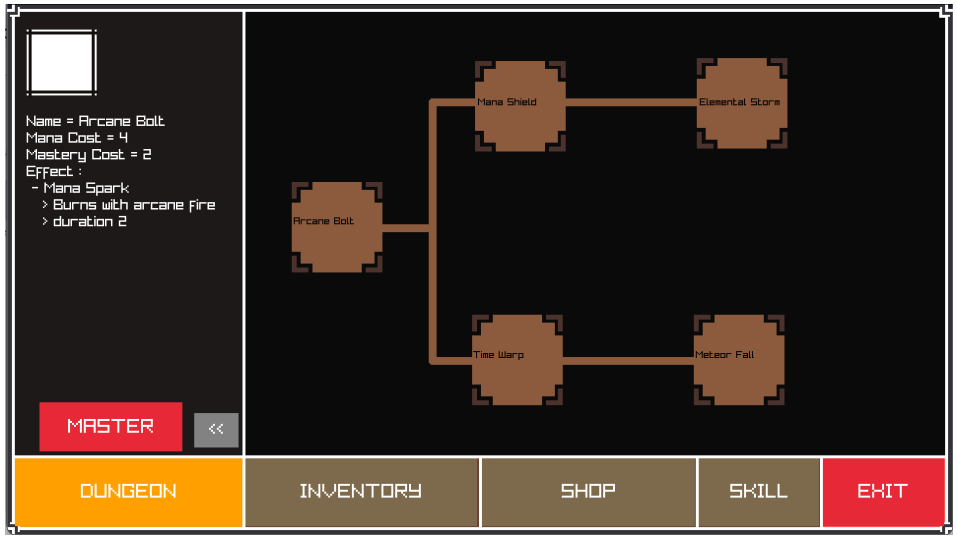
Kelas SaveScreen berfungsi sebagai layar untuk menyimpan konfigurasi ke folder tertentu.

3.1.1.6.11. ShopScreen



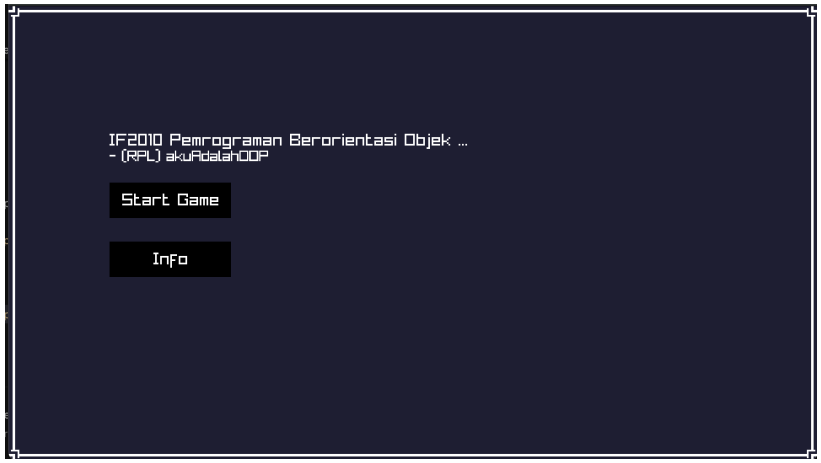
Kelas ShopScreen berfungsi untuk menampilkan menu toko yang dapat diakses oleh pengguna. Layar ini memungkinkan pemain membeli item yang tersedia di dalam game, serta menampilkan informasi terkait harga dan ketersediaan barang.

3.1.1.6.12. SkillPathScreen



Kelas SkillPathScreen berfungsi untuk menampilkan menu jalur keterampilan (skill path) yang dapat dipilih dan digunakan oleh pemain. Layar ini memperlihatkan pohon keterampilan, memungkinkan pengguna menavigasi, memilih, dan meng-upgrade kemampuan karakter sesuai progres permainan.

3.1.1.6.13. StartScreen



Kelas StartScreen berfungsi sebagai tampilan awal ketika pengguna menjalankan aplikasi. Layar ini menyajikan opsi untuk memulai permainanatau mengakses informasi.

3.2. Bonus Kreasi Mandiri

3.2.1 *Sound Effect*

Cuplikan kode playsound untuk *sound effect*
`hoverSfx = LoadSound("./Assets/sfx/ui/hover.wav");` `clickSfx = LoadSound("./Assets/sfx/ui/confirm.wav");`

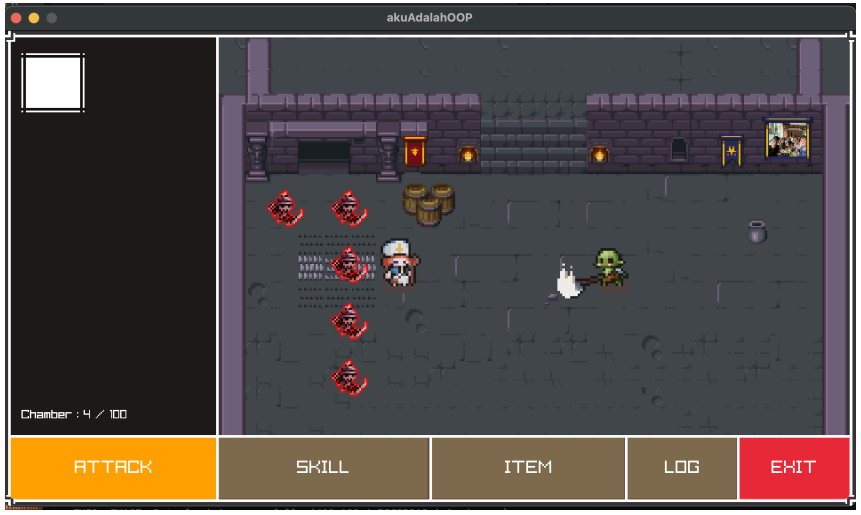
```
int dmg = getGameState().getCharacter()->attack(selectedUnit);
PlaySound(atkSound);
if (!isAction)
{
```

Penjelasan :

Sebagai bonus kreasi, kami menambahkan *sound effect* untuk membuat permainan terasa lebih hidup dan interaktif. Efek suara ini muncul saat aksi-aksi penting terjadi, seperti serangan, terkena damage, atau penggunaan skill, sehingga pemain tidak hanya melihat aksi di layar, tapi juga bisa merasakannya lewat suara. Fitur ini menambah keseruan dan membuat pertarungan jadi lebih seru dan imersif, tanpa mengganggu performa permainan.

3.2.2 *Animation Attack*

Gambar :

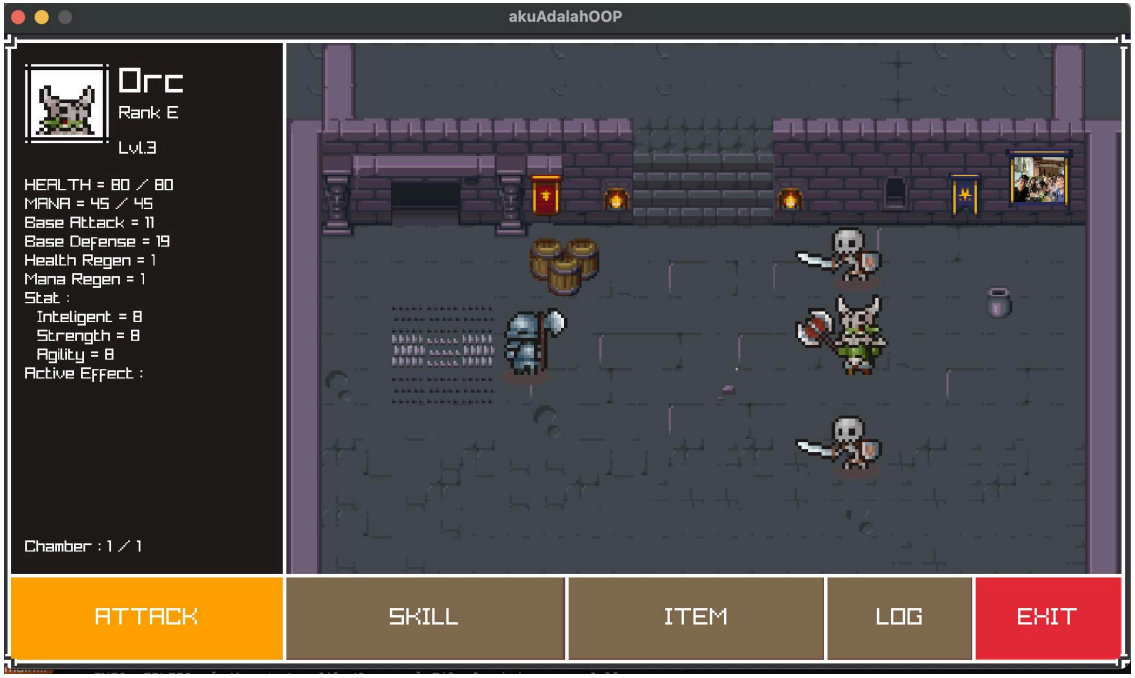


Penjelasan :

Kelompok kami juga mengimplementasikan fitur *attack animation* untuk memperkaya pengalaman visual selama pertarungan berlangsung. Animasi ini muncul setiap kali karakter melakukan serangan, memberikan efek gerakan atau visual yang merepresentasikan aksi tersebut, sehingga pertarungan tidak hanya ditampilkan melalui teks, tetapi juga disertai elemen visual yang menarik. Fitur ini disinkronkan dengan logika battle yang sudah ada dan dirancang agar tetap ringan serta tidak mengganggu performa program. Penambahan ini membuat proses pertarungan menjadi lebih hidup, interaktif, dan menyenangkan bagi pengguna.

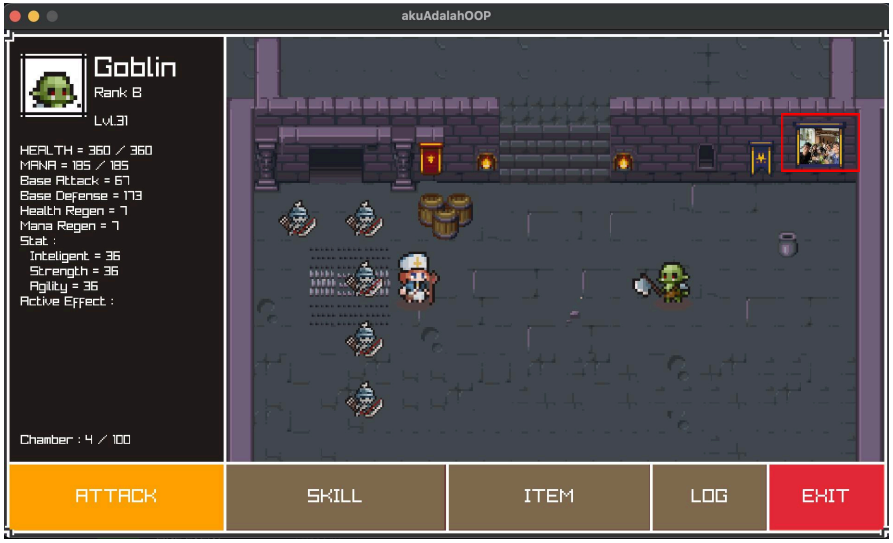
3.2.3 *One vs Many Mob*

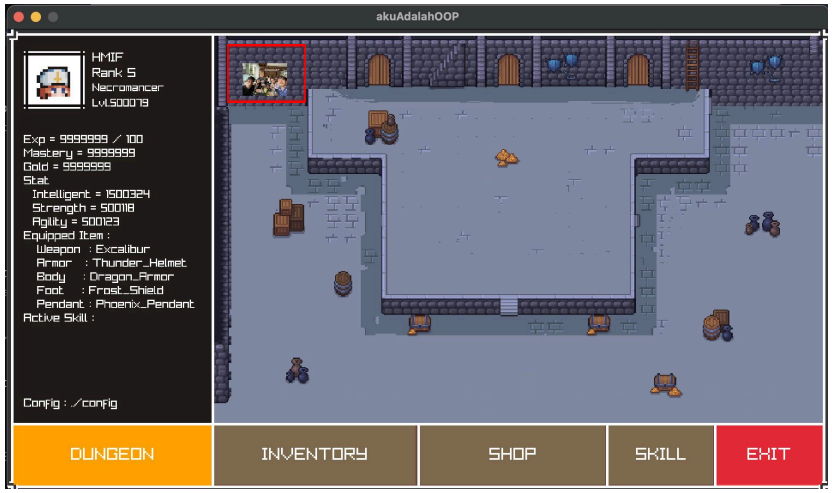
Gambar :



Penjelasan :
Sebagai bonus kreasi, kami menambahkan fitur *One vs Many Mob* yang memungkinkan satu karakter melawan lebih dari satu mob dalam satu chamber. Dalam spesifikasi wajib, setiap chamber hanya berisi satu musuh, namun kami memodifikasi sistem pertarungan agar mendukung skenario dimana pemain harus menghadapi beberapa mob sekaligus. Hal ini membuat permainan menjadi lebih menantang dan seru.

3.2.4 *Easter Egg*

Gambar :




Penjelasan :

Terdapat foto wajah kelompok kami pada dungeon Home Screen. Yang lebih Easter Egg lagi, hanya ada 4 orang di foto itu (Kelompok kami ada 5 orang). Absolute Cinema !!!!!



Source: https://images.app.goo.gl/f9LZT87uFEdvmG6R9

4. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Unit	13523091	13523073, 13523091, 13523107, 13523121
Character	13523091	13523073, 13523091, 13523107, 13523121
Skill dan Skill Path	13523121, 13523073, 13523091	13523073, 13523091, 13523107, 13523121
Effect	13523091, 13523107	13523073, 13523091, 13523107, 13523121
Dungeon	13523073	13523073, 13523091, 13523107, 13523121
Mobs	13523091	13523073, 13523091, 13523107, 13523121
Shop	13523107	13523073, 13523091, 13523107, 13523121
Items	13523107	13523073, 13523091, 13523107, 13523121
Damage	13523091	13523073, 13523091, 13523107, 13523121
Inventory	13523107	13523073, 13523091, 13523107, 13523121

Save/Load	13523073, 13523107	13523073, 13523091, 13523107, 13523121
Antarmuka Pengguna Grafis (APG)	13523073,13523121	13523073, 13523091, 13523107, 13523121

Form Asistensi Tugas Besar
IF2010/Pemrograman Berorientasi Objek
Sem. 2 2024/2025

No. Kelompok/Kode Kelompok : RPL
Nama Kelompok : akuAdalahOOP
Anggota Kelompok (Nama/NIM) :
1. Kharris Khisunica / 13522051
2. Alfian Hanif Fitria Yustanto / 13523073
3. Carlo Angkisan / 13523091
4. Heleni Gratia M Tampubolon / 13523107
5. Ahmad Wicaksono / 13523121

Asisten Pembimbing : Nicholas Liem (13521135)

Tanggal : 22 April 2024 **Tempat :** Google Meet	**Catatan Asistensi:** **Pertanyaan:** Bagaimana konsep dari **restock**? Mengapa terdapat probabilitas pada penjelasan di spesifikasinya? **Jawaban:** Probabilitas dihapuskan pada konsep restock tersebut. Cukup restock sesuai config saja atau dapat menerapkan sistem random. **Pertanyaan:** Di bagian **Item**, pada gambar terlihat nama yang sama namun rarity berbeda. Bagaimana maksudnya? **Jawaban:** Misalnya saja, pedang dapat a, b, c, namun tipenya harus berbeda-beda. Silakan sesuaikan dengan kreativitas saja. **Pertanyaan:** Bagaimana maksud dari **double dungeon**? apakah ada dungeon di dalam dungeon? **Jawaban:** Maksudnya adalah tingkat kesulitan yang lebih tinggi, seperti musuh yang semakin sulit untuk dikalahkan. Contohnya, dikalikan 1.5 untuk stats musuh. Jadi, bukan menambah chamber. **Pertanyaan:** **Mob** memiliki level, lalu bagaimana stats pada level tersebut? Apakah dikreasikan sendiri? **Jawaban:** Iya, silakan kreasikan sendiri. **Pertanyaan:** Terkait **skill dan skillpath**, jika pada awalnya karakter memiliki 1 skill lalu dapat upgrade, skill yang dimiliki sekarang ditentukan berdasarkan apa? **Jawaban:** Skill awal (base skill) yang dimiliki akan berdasarkan characternya. Base skill tidak hilang. Namun, jika di upgrade base skill akan hilang. Jika ada cabang di keluarga skill yang sama, path yg tidak dipilih tidak dapat diakses lagi.

Kehadiran Anggota Kelompok:

1
Kharris Khisunica
Tanda tangan

2
Alfian Hanif Fitria Yustanto
13523073

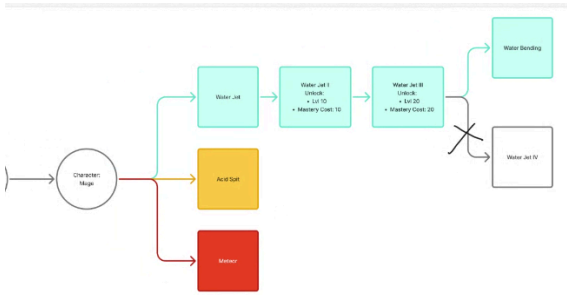


3
Carlo Angkisan
13523091



4
Heleni Gratia M Tampubolon
13523107





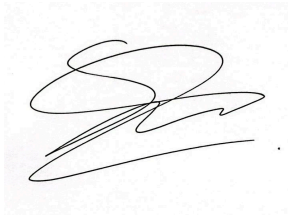
Pertanyaan:
Saat character berada di dungeon, jika ia ingin mengakses **inventory** apakah diperbolehkan secara bebas? Kalau mau mengganti senjata apakah diperbolehkan juga? Atau hanya sekedar memakai item saja?
Jawaban:
Bebas saja sesuai dengan kreativitas.

Pertanyaan:
Apakah yang dijual di shop harus berdasarkan **config** shot.txt?
Jawaban:
Iya, sehingga tidak bisa menjual selain di luar config. Kecuali ingin mengimplementasikan sendiri.

Pertanyaan:
Pada **character** Necromancer terdapat anak buahnya. Apakah cara menyerangnya dibebaskan?
Jawaban:
Untuk turn as a player atau turn as a anak buah dibebaskan saja. Anak buah Necromancer juga memiliki hp tersendiri.

Pertanyaan:
Untuk implementasi **GUI** apakah dibebaskan?
Jawaban:
GUInya dibebaskan selagi tidak memakai game engine. GUI hanya dapat mengatur tampilan saja dan tidak boleh mengatur game state.

Pertanyaan:
Jika untuk membuat **Main** susah, kemudian ditambahkan **kelas pembantu** apakah boleh?
Jawaban:
Silakan saja, gunakan nama ‘game state’ untuk mudah dipahami. Sehingga dapat menggunakan game state untuk menyimpan state dari game saja.

5 Ahmad Wicaksono 13523121 	
	Tanda Tangan Asisten:  Nicholas Liem (13521135)